Industrie- und Handelskammer
für Ostfriesland und Papenburg

Bestätigung über die durchgeführte Projektarbeit im IT- Ausbildungsberuf

Fachinformatiker für Anwendungsentwicklung

Abschlussprüfung: Sommer 2026 Winter

Azubi-Identnummer: 121/ 572525

Ausbildungsfirma:

Name : Hamburg

Vorname: Niklas

Straße: Kapitän-Rüter-Str. 81

PLZ / Ort: 26871 Papenburg

Firma : journaway GmbH

Straße : Mühlenstr. 111

PLZ / Ort: 26789 Leer

Projektbezeichnung: Bereitstellung eines Microservices zur Synchronisation von dezentralisiert gespeicherten Kundendaten

Projektbeginn: 29.03.2026 Projektfertigstellung: 05.05.2026 Zeitaufwand in Std. 80

Bestätigung der Ausbildungsfirma:

Wir bestätigen, dass der Prüfling das oben bezeichnete Projekt einschließlich der Dokumentation im oben genannten Zeitraum selbständig ausgeführt hat.

Leer, 05.05.2026 — TStamma

Ort, Datum — Unterschrift Ausbildender

Alicante, 05.05.2026

Ort, Datum — Unterschrift Projektverantwortliche/r

Eidesstattliche Erklärung des Prüflings:

Ich versichere, dass ich die Fachaufgabe selbständig durchgeführt und den dazugehörenden Report selbständig erstellt habe.

Papenburg, 05.05.2026 — N.Hamburg

Ort, Datum — Unterschrift Prüfling

Diese Bestätigung ist als 1. Seite der Dokumentation hochzuladen!

Abschlussprüfung Sommer 2026

Fachinformatiker für Anwendungsentwicklung

Dokumentation zur betrieblichen Projektarbeit

Bereitstellung eines Microservices

zur Synchronisation von dezentralisiert gespeicherten Kundendaten

Abgabetermin: Papenburg, den 08.05.2026

Prüfungsbewerber:
Niklas Hamburg
Kapitän-Rüter-Straße 81
26871 Papenburg

Ausbildungsbetrieb:
JOURNAWAY journaway GmbH
Mühlenstr. 111
26789 Leer

Inhaltsverzeichnis

Abbildungsverzeichnis **III**

Tabellenverzeichnis **IV**

Abkürzungsverzeichnis **V**

1 Einleitung **1**
1.1 Projektumfeld . . . 1
1.2 Projektziel . . . 1
1.3 Projektbegründung . . . 2
1.4 Projektschnittstellen . . . 2
1.5 Projektabgrenzung . . . 2

2 Projektplanung **3**
2.1 Projektphasen . . . 3
2.2 Abweichungen vom Projektantrag . . . 3
2.3 Ressourcenplanung . . . 3
2.4 Entwicklungsprozess . . . 4

3 Analyse **4**
3.1 Ist-Analyse . . . 4
3.2 Wirtschaftlichkeitsanalyse . . . 5
3.3 Anforderungsanalyse . . . 6
3.4 Nutzwertanalyse . . . 7

4 Entwurf **7**
4.1 UML-Aktivitätsdiagramm . . . 8
4.2 Architekturdesign . . . 8
4.3 Datenbankstruktur . . . 9

5 Implementierung **9**
5.1 Implementierung der Datenstrukturen . . . 9
5.2 Implementierung RabbitMQ . . . 10
5.3 Implementierung des HTTP-Clients . . . 10
5.4 Implementierung der Synchronisationshistorie . . . 11
5.5 Implementierung der Geschäftslogik . . . 11

6 Abnahmephase **12**
6.1 Unittests . . . 12
6.2 Integrationstests . . . 12
6.3 Test Ergebnisse . . . 13

7 Deployment **13**

8 Dokumentation **13**
8.1 Entwicklerdokumentation 14

9 Fazit **14**
9.1 Soll-/Ist-Vergleich 14
9.2 Lessons Learned 15
9.3 Ausblick 15

Literaturverzeichnis **16**

A Anhang **i**
A.1 Detaillierte Zeitplanung i
A.2 Jira-Tickets ii
A.3 Synchronisations Abläufe iii
A.4 Architektur iv
A.5 Datenbankmodell v
A.6 RabbitMQ vi
A.7 HTTP-Client vii
A.8 Synchronisationshistorie viii
A.9 Geschäftslogik ix
A.10 Tests x
A.11 Pipelines und Automatisierung xiii
A.12 Entwicklerdokumentation xiv

Abbildungsverzeichnis

1 Ticket Synchronisation . . . ii
2 Ticket Datenbank und RabbitMQ . . . ii
3 Ticket Pipelines . . . ii
4 Vereinfachte Architekturübersicht der Nightcrawler-Solution mit Orchestrator-Projekt und Anwendungsschichten . . . iv
5 Nightcrawler ER-Diagramm . . . v
6 RabbitMQ DI-Registrierung . . . vi
7 Queue-Initialisierung . . . vi
8 Event Publisher . . . vi
9 Event Consumer . . . vi
10 HubSpotApiClient . . . vii
11 HubSpotApiRateLimiter . . . vii
12 Drei Abstraktionsebenen zur Persistierung der Synchronisationshistorie . . . viii
13 DbOperation . . . viii
14 Repository . . . viii
15 Service . . . viii
16 PersonalInformation Upsert . . . ix
17 API-Logik . . . ix
18 RegEx-Aufruf . . . ix
19 CommunicationPreference-Logik . . . ix
20 Nightcrawler Projekt Struktur . . . x
21 Testabdeckung . . . x
22 Unit Test . . . xi
23 Integrationstest . . . xii
24 Tests und Codeanalyse . . . xiii
25 Docker Images . . . xiii
26 Versionierung . . . xiii
27 Deployment per SSH . . . xiii
28 Ausschnitt Entwicklerdokumentation . . . xiv

Tabellenverzeichnis

1 Zeitplanung 3
2 Nutzwertvergleich 7
3 Soll-/Ist-Vergleich 15

Abkürzungsverzeichnis

API	Application Programming Interface
ORM	Object-Relational Mapping
CRM	Customer Relationship Management
DLQ	Dead-Letter-Queue
TTL	Time-To-Live
ERD	Entity-Relationship-Diagramm
EFCore	Entity-Framework-Core
DI	Dependency Injection
URL	Uniform Resource Locator
RegEx	Regular Expression
Json	JavaScript Object Notation
SSH	Secure Shell

1 Einleitung

In dieser Dokumentation wird das Projekt „Bereitstellung eines Microservices zur Synchronisation von dezentralisiert gespeicherten Kundendaten“ beschrieben, welches im Rahmen der Prüfung zum Abschluss der Ausbildung zum Fachinformatiker für Anwendungsentwicklung durchgeführt wurde. Es wird der Prozess von der Analyse der Ausgangssituation über die Bewertung möglicher Lösungsansätze bis hin zur Umsetzung einer geeigneten Anwendung dargestellt.

1.1 Projektumfeld

Die journaway GmbH ist ein in Leer ansässiges, 2017 gegründetes Reiseunternehmen und bietet Reisekomplettpakete über ihre Webseite an. Derzeit beschäftigt das Unternehmen rund 160 Mitarbeiter. Mehr als ein Drittel davon arbeitet in der internen IT-Abteilung, welche B2C- und B2B-Lösungen für das Unternehmen bereitstellt.

Zu den Aufgaben dieser Abteilung gehören unter anderem die Entwicklung und Pflege der Webseite sowie der dazugehörigen Daten, beispielsweise Buchungsdaten, Kundendaten, Preise und Leistungsinformationen. Dadurch soll sichergestellt werden, dass Kunden und interne Mitarbeiter jederzeit auf aktuelle Informationen zugreifen können.

Der Auftraggeber des Projektes ist die journaway GmbH selbst.

1.2 Projektziel

Die journaway GmbH hat in Auftrag gegeben, eine geeignete Lösung zu finden, um Kundendaten aus zwei getrennten Systemen miteinander zu synchronisieren. Dabei soll im Projektverlauf geprüft werden, ob eine Eigenentwicklung sinnvoll ist oder ob eine bestehende Lösung eingekauft und integriert werden kann.

Mit der Entwicklung eines neuen Kundenportals auf der Webseite ist es notwendig, dass kundenbezogene Informationen für die Kommunikation zwischen Unternehmen und Kunden nicht uneinheitlich in verschiedenen Systemen vorliegen. Ziel des Projektes ist es daher, eine Lösung bereitzustellen, durch die relevante Kundendaten automatisch abgeglichen und konsistent gehalten werden können.

Die Lösung soll dazu beitragen, widersprüchliche Datenbestände zu vermeiden, den manuellen Pflegeaufwand zu reduzieren und eine verlässliche Grundlage für Prozesse im Kundenportal sowie im bestehenden CRM-System zu schaffen.

1.3 Projektbegründung

In den bisherigen Systemen werden Kundendaten unabhängig voneinander gespeichert. Da zwischen diesen Datenbeständen bislang keine durchgängige technische Verknüpfung besteht, können Informationen zum gleichen Kunden unterschiedlich vorliegen. Dies betrifft insbesondere kontaktrelevante Daten wie E-Mail-Adresse, Vorname, Nachname, Anrede und Kommunikationspräferenzen.

Fehlerhafte oder veraltete Kundendaten können dazu führen, dass Informationen im Kundenportal nicht korrekt dargestellt werden oder die Kommunikation mit dem Kunden auf Grundlage falscher Daten erfolgt. Außerdem erschwert die fehlende Verknüpfung zwischen den Systemen, Kundendaten aus dem CRM-System eindeutig einem angemeldeten Kunden im Kundenportal zuzuordnen.

Ohne eine automatisierte Lösung müssten Mitarbeiter aus dem Kundensupport Abweichungen in Kontaktinformationen manuell erkennen, prüfen und korrigieren. Dies würde zusätzlichen Ressourcenaufwand verursachen und die Fehleranfälligkeit erhöhen. Ziel ist es daher, aktuelle und konsistente Kontaktdaten ohne regelmäßigen manuellen Pflegeaufwand sicherzustellen.

1.4 Projektschnittstellen

Die zu betrachtende Lösung muss mit zwei bestehenden Systemen zusammenarbeiten. Das erste System ist der *user-service*, welcher Daten des Kundenportals verwaltet. Dazu gehören unter anderem persönliche Informationen, Kommunikationspräferenzen, Reisende und Buchungsinformationen angemeldeter Webseitenbenutzer.

Das zweite System ist das CRM-System *HubSpot*. Dort werden kundenbezogene Daten wie Kontaktdaten, Deals und Informationen für Workflows gespeichert. HubSpot wird im Unternehmen insbesondere für CRM-Prozesse genutzt.

Da die spätere Lösung der Synchronisation zwischen bestehenden Systemen dient, gibt es keine direkten Benutzer der Anwendung. Änderungen an Kundendaten entstehen entweder durch Nutzeraktionen im Kundenportal oder durch Mitarbeiter im CRM-System. Im Projektverlauf ist festzulegen, über welche technischen Schnittstellen diese Änderungen verarbeitet und an das jeweils andere System übertragen werden.

1.5 Projektabgrenzung

Teil dieses Projektes ist die Analyse, Planung und Entwicklung einer funktionalen und testbaren Lösung zur Synchronisation von Kundendaten. Die Aufnahme der Anwendung in den Livebetrieb wird nicht behandelt.

2 Projektplanung

Um einen strukturierten Ablauf des Projektes zu gewährleisten, werden in diesem Abschnitt die Projektphasen, Abweichungen vom Projektantrag, benötigte Ressourcen sowie der Entwicklungsprozess beschrieben.

2.1 Projektphasen

Die Umsetzung des Projektes erfolgt innerhalb von zwei vierwöchigen Sprints. Gemäß den Vorgaben der IHK wird das Projekt, wie in Tabelle 1 dargestellt, innerhalb von 80 Stunden bearbeitet.

Zeitplanung Tabelle 1 zeigt die grobe Zeitplanung.

Projektphase	Geplante Zeit
Analyse	8 h
Entwurf	6 h
Implementierung	49 h
Einführung	7 h
Dokumentation	10 h
Gesamt	**80 h**

Tabelle 1: Zeitplanung

Eine detailliertere Zeitplanung findet sich im Anhang A.1: Detaillierte Zeitplanung auf Seite i.

2.2 Abweichungen vom Projektantrag

Im Vergleich zum Projektantrag wurde die geplante Anbindung von Hangfire durch die Anbindung an RabbitMQ ersetzt. Während der Planung wurde festgestellt, dass Hangfire für die erste Iteration nicht benötigt wird. Da die Synchronisation ereignisbasiert erfolgen soll, wurde stattdessen RabbitMQ als Message Broker vorgesehen.

2.3 Ressourcenplanung

Für die Durchführung des Projektes werden ein Büroarbeitsplatz mit Internetzugang, ein Entwicklungsgerät mit zusätzlichen Bildschirmen sowie Serverressourcen für Anwendung und Datenbank benötigt. Als Software kommen Windows 11, JetBrains Rider 2026, RabbitMQ, PostgreSQL 16, Debian 12.13 sowie HeidiSQL und pgAdmin 4 zur Datenbankverwaltung zum Einsatz.

Zusätzlich werden während der Review- und Testphase zwei weitere Entwickler zur Qualitätssicherung einbezogen.

2.4 Entwicklungsprozess

Während der Umsetzung wird ein agiler Entwicklungsprozess nach Scrum verwendet. Das Projekt wird dabei in kleinere Arbeitspakete aufgeteilt, die innerhalb der Sprints umgesetzt werden. Dadurch kann flexibel auf technische Probleme oder geänderte Anforderungen reagiert werden.

Die Planung und Nachverfolgung der Aufgaben erfolgt über Tickets auf einem Jira-Board. Die Tickets beschreiben die einzelnen Arbeitspakete und dienen während des Projektes zur Fortschrittskontrolle. Eine Übersicht der verwendeten Tickets befindet sich im Anhang A.2: Jira-Tickets auf Seite ii.

3 Analyse

Im Folgenden wird zunächst die bisherige Situation im Unternehmen betrachtet. Darauf aufbauend wird untersucht, welchen wirtschaftlichen Nutzen das Projekt bietet. Anschließend werden die fachlichen und technischen Anforderungen an die Anwendung beschrieben. Zum Abschluss folgt eine Nutzwertanalyse, um den nicht-monetären Nutzen des Projektes zu bewerten.

3.1 Ist-Analyse

Aktuell werden kundenbezogene Daten in zwei voneinander getrennten Systemen gespeichert. Zum einen im *user-service*, welcher Daten für das Kundenportal verwaltet, und zum anderen im CRM-System *HubSpot*. Zwischen beiden Systemen besteht bislang keine eigenständig verwaltete und durchgängig nachvollziehbare Synchronisation. Dadurch kann es passieren, dass Kontaktinformationen eines Kunden in beiden Systemen unterschiedlich vorliegen.

Besonders problematisch ist dies bei Informationen, die für die Kommunikation mit dem Kunden relevant sind. Dazu zählen unter anderem die E-Mail-Adresse, der Vorname, der Nachname, die Anrede sowie Kommunikationspräferenzen. Werden diese Daten in einem der beiden Systeme geändert, ist nicht sichergestellt, dass die Änderung zuverlässig im jeweils anderen System übernommen wird.

Zusätzlich existiert bislang keine zentrale Stelle, die nachvollziehbar speichert, wann eine Synchronisation durchgeführt wurde, aus welchem System eine Änderung stammt und ob diese erfolgreich verarbeitet wurde. Fehler bei der Übertragung könnten dadurch nur mit erhöhtem manuellem Aufwand erkannt und korrigiert werden. Ebenso gibt es bisher keine eigenständige technische Lösung, um mit temporären Fehlern, Systemausfällen oder Rate Limits der HubSpot API umzugehen.

Aus diesem Grund soll mit dem Projekt *Nightcrawler* ein eigenständiger Microservice entwickelt werden, der die Synchronisation zwischen beiden Systemen übernimmt. Der Name ist an den teleportierenden Helden angelehnt, da die Anwendung Kundendaten kontrolliert zwischen zwei voneinander getrennten Systemen überträgt. Hierzu soll eine PostgreSQL-Datenbank eingeführt werden, in der

sowohl der aktuelle Synchronisationszustand zwischen Benutzer und Kontakt als auch die Historie aller Synchronisationsereignisse gespeichert wird The PostgreSQL Global Development Group [2026].

Die Kommunikation zwischen den beteiligten Systemen soll asynchron über RabbitMQ erfolgen. Die bisherige Situation wird dadurch verbessert, dass Kundendaten automatisch, nachvollziehbar und fehlertolerant synchronisiert werden können. Zusätzlich soll die neue Anwendung Konflikte bei gleichzeitigen Änderungen erkennen und nach festgelegten Regeln auflösen.

3.2 Wirtschaftlichkeitsanalyse

Für die Umsetzung des Projektes sind insgesamt 80 Stunden vorgesehen. Bei einem internen Verrechnungssatz von 35 Euro Netto pro Stunde ergeben sich folgende Entwicklungskosten:

$$80 \text{ Stunden} \times 35 \text{ Euro/Stunde} = 2800 \text{ Euro}$$

Dem gegenüber steht der Nutzen der automatisierten Synchronisation. Ohne eine solche Lösung müssten Abweichungen zwischen den Datenbeständen manuell erkannt, überprüft und korrigiert werden. Dies würde insbesondere im Kundensupport und in der technischen Betreuung zusätzlichen Aufwand verursachen.

Wird angenommen, dass durch die automatisierte Synchronisation wöchentlich 2,5 Stunden manueller Aufwand eingespart werden können, ergibt sich bei einem internen Stundensatz von 15,50 Euro Netto pro Stunde ein jährliches Einsparpotenzial von:

$$2{,}5 \text{ Stunden/Woche} \times 15{,}50 \text{ Euro/Stunde} \times 52 \text{ Wochen} = 2015 \text{ Euro/Jahr}$$

Die Amortisationsdauer der Eigenentwicklung beträgt damit:

$$2800 \text{ Euro} \div 2015 \text{ Euro/Jahr} \approx 1{,}39 \text{ Jahre}$$

Die Entwicklungskosten sind somit nach ungefähr 17 Monaten durch die eingesparte Arbeitszeit ausgeglichen.

Im Rahmen der Make-or-Buy-Entscheidung wurde außerdem geprüft, ob eine bestehende Integrationsplattform oder ein fertiger HubSpot-Connector eingesetzt werden kann. Eine solche Buy-Lösung hätte zwar einen geringeren Entwicklungsaufwand, würde jedoch laufende Lizenzkosten verursachen und könnte die projektspezifischen Anforderungen nur eingeschränkt erfüllen. Dazu zählen insbesondere die eigene Synchronisationshistorie, die definierte Konfliktbehandlung, die Retry-Logik sowie die enge Integration in die bestehende Systemarchitektur.

Für eine Buy-Lösung wird beispielhaft ein einmaliger Einrichtungsaufwand von 16 Stunden sowie ein monatlicher Lizenzbetrag von 250 Euro Netto angesetzt:

$$16 \text{ Stunden} \times 35 \text{ Euro/Stunde} + 250 \text{ Euro/Monat} \times 12 \text{ Monate} = 3560 \text{ Euro}$$

Damit liegen die Kosten der Buy-Lösung bereits im ersten Jahr über den Entwicklungskosten der Eigenentwicklung. Zusätzlich bietet die Eigenentwicklung einen höheren fachlichen Nutzen und eine bessere Erweiterbarkeit. Aus wirtschaftlicher und fachlicher Sicht wird daher die Eigenentwicklung des *Nightcrawlers* umgesetzt.

3.3 Anforderungsanalyse

Damit das Projektziel erreicht werden kann, muss die Anwendung sowohl fachliche als auch technische Anforderungen erfüllen.

Fachlich muss der *Nightcrawler* in der Lage sein, Änderungen an Kundendaten zwischen dem *user-service* und *HubSpot* in beide Richtungen zu synchronisieren. Dabei sollen mindestens die Datenfelder *email*, *givenName*, *surname*, *salutation* und *CommunicationPreference* berücksichtigt werden.

Wenn ein Benutzer im *user-service* erstellt oder geändert wird, soll ein entsprechendes Ereignis verarbeitet und die dazugehörigen Daten in *HubSpot* aktualisiert werden. Wird ein Benutzer neu erstellt, soll zunächst geprüft werden, ob anhand der E-Mail-Adresse bereits ein passender Kontakt in *HubSpot* existiert. Ist dies der Fall, soll dieser Kontakt aktualisiert werden. Falls kein Kontakt gefunden wird, soll ein neuer Kontakt erstellt werden.

Wenn dagegen in *HubSpot* ein Kontakt geändert wird, soll diese Änderung ebenfalls verarbeitet und an den *user-service* weitergegeben werden. Kontakte aus *HubSpot*, für die kein passender Benutzer im *user-service* existiert, sollen zunächst ignoriert werden.

Damit jede Synchronisation nachvollziehbar bleibt, muss jedes verarbeitete Ereignis zunächst in einer Historie gespeichert werden. Diese *SyncHistory* soll Informationen zur Quelle des Ereignisses, zum Ereignistyp, zu Benutzer- und Kontaktzuordnung, zum Payload, zum Zeitpunkt des Auftretens, zum aktuellen Status sowie zur Anzahl der bisherigen Verarbeitungsversuche enthalten.

Zusätzlich muss ein *UserContactState* gespeichert werden. Dieser dient dazu, die Zuordnung zwischen Benutzer und HubSpot-Kontakt sowie den letzten bekannten Synchronisationsstand zu verwalten. Dabei sollen unter anderem die Zeitpunkte der letzten Änderung im *user-service*, der letzten Änderung in *HubSpot* und der letzten erfolgreichen Synchronisation gespeichert werden.

Technisch muss die Anwendung acht Event Queues über RabbitMQ verwenden. Da keine Nachrichten bei einem Neustart oder Ausfall verloren gehen sollen, müssen die Hauptqueues als durable Quorum

Queues umgesetzt werden. Für temporäre Fehler soll eine Retry-Logik mit TTL-basierten Retry Queues eingesetzt werden. Nicht behandelbare Fehler sollen in einer Dead-Letter-Queue abgelegt werden Broadcom Inc. [2026].

Zur Konfliktbehandlung soll eine optimistische Strategie verwendet werden. Wenn Daten in beiden Systemen unterschiedlich geändert wurden, soll der Datenstand des *user-service* bevorzugt werden. Wird also eine Änderung aus *HubSpot* erkannt, die im Konflikt mit aktuelleren Daten aus dem *user-service* steht, soll diese Änderung nicht übernommen, sondern stattdessen eine neue Synchronisation in Richtung *HubSpot* angestoßen werden.

3.4 Nutzwertanalyse

Neben dem wirtschaftlichen Nutzen bietet das Projekt auch einen nicht-monetären Mehrwert. Zur Bewertung werden die Kriterien Datenkonsistenz, Nachvollziehbarkeit, Ausfallsicherheit, Wartbarkeit und Automatisierungsgrad betrachtet. Dabei werden die bisherige Ist-Situation, eine mögliche Buy-Lösung und die Eigenentwicklung des *Nightcrawlers* verglichen.

Kriterium	Gewichtung	Ist-Situation	Buy-Lösung	Eigenentwicklung
Datenkonsistenz	5	2	3	5
Nachvollziehbarkeit	3	1	3	5
Ausfallsicherheit	3	2	3	4
Wartbarkeit	2	2	3	4
Automatisierungsgrad	2	2	4	4
Gesamt:	**15**	**27**	**47**	**68**
Nutzwert:		**1,8**	**3,13**	**4,53**

Tabelle 2: Nutzwertvergleich

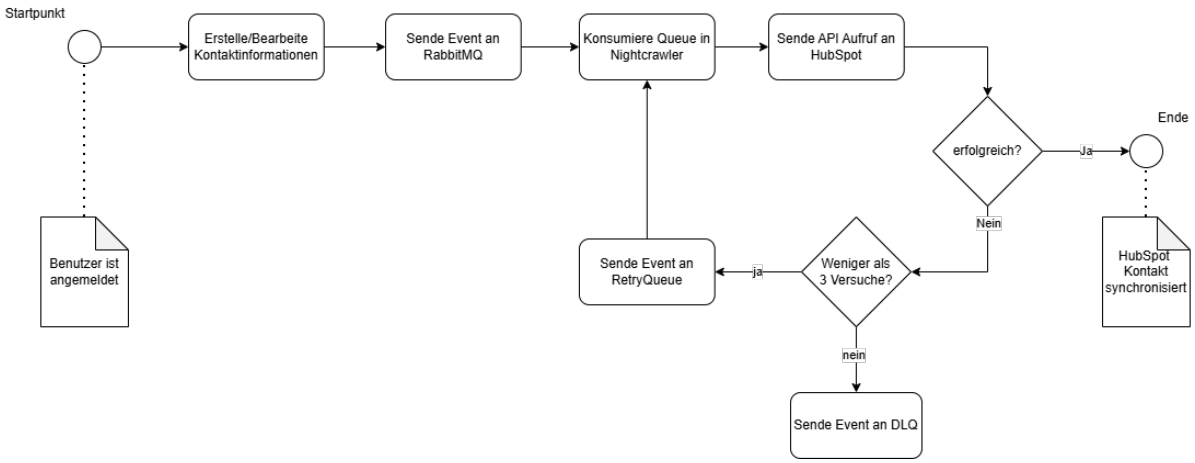
Aus der Nutzwertanalyse ergibt sich, dass eine Buy-Lösung gegenüber der bisherigen Situation zwar eine Verbesserung darstellt, jedoch nicht alle projektspezifischen Anforderungen vollständig erfüllt. Die Eigenentwicklung erreicht den höchsten Nutzwert, da sie besonders bei Nachvollziehbarkeit, Konfliktbehandlung, Ausfallsicherheit und Anpassbarkeit an die bestehende Systemarchitektur Vorteile bietet. Die Entscheidung zugunsten des *Nightcrawlers* wird dadurch zusätzlich bestätigt.

4 Entwurf

In dieser Phase soll die Implementierung vorbereitet werden. Dazu wird in einem UML-Aktivitätsdiagramm dargestellt, wie der Synchronisationsablauf in eine Richtung aussieht. Anschließend zeigt eine Architekturübersicht, wie diese Systeme miteinander integagieren. Am Ende stellt ein ER-Modell dar, wie die Datenbankstruktur für die Event Persistierung aussieht.

4.1 UML-Aktivitätsdiagramm

Das folgende UML-Aktivitätsdiagramm für die Synchronisation in Richtung *HubSpot*, zeigt den Ablauf der Synchronisation vom *user-service* ausgehend. Angefangen mit der Eingabe der Kontaktdaten vom Nutzer wird ein Event in die Main Queue geschickt. Anschließend konsumiert der *Nightcrawler* diese Queue und sendet basierend auf den Daten innerhalb der Event Message, einen API Upsert Aufruf an *HubSpot*. Sofern der API Aufruf einen 201 Created oder 200 Ok Statuscode zurückliefert wird das Event als erfolgreich angesehen. Andernfalls wird das Event in eine Retry Queue geschickt und nach einer 30 sekündigen TTL beim ersten Mal und 60 Sekunden beim zweiten Mal erneut ausgeführt. Wenn das zweite Mal ebenfalls nicht erfolgreich ist, wird das Event in eine DLQ geschickt.



Für genauere technische Abläufe siehe Anhang A.3: Synchronisations Abläufe auf Seite iii

4.2 Architekturdesign

Die Anwendung ist als mehrschichtige Microservice-Architektur aufgebaut. Innerhalb der Nightcrawler-Solution existiert ein eigenes Orchestrator-Projekt, welches den Start der Anwendung, die Konfiguration sowie die Registrierung der benötigten Services übernimmt. Dadurch werden technische Abhängigkeiten zentral verwaltet und die einzelnen Anwendungskomponenten klar voneinander getrennt.

Die fachliche Verarbeitung ist in mehrere Schichten aufgeteilt. Eingehende Ereignisse werden zunächst von der *Presentation Layer* konsumiert und anschließend an die Anwendungsschicht weitergegeben. Dort wird der Synchronisationsablauf koordiniert. Die *Domain Layer* enthält die fachlichen Modelle und Regeln, während die *Infrastructure Layer* die technischen Anbindungen an externe Systeme wie RabbitMQ, HubSpot und PostgreSQL bereitstellt.

Abbildung 4 zeigt den vereinfachten Aufbau der Nightcrawler-Solution mit Orchestrator-Projekt, Anwendungsschichten und externen Schnittstellen.

Die Anwendung wird mit Microsoft .NET 10 und C# entwickelt. Diese Technologie wurde gewählt, da alle Backend-Services im Unternehmen auf C# und .NET basieren. Dadurch kann die neue Anwendung einheitlich in die bestehende Systemlandschaft integriert und von den vorhandenen Entwicklern gewartet werden.

Ein weiterer Grund für den Einsatz von .NET ist die plattformunabhängige Laufzeitumgebung. Die Anwendung kann dadurch sowohl lokal während der Entwicklung als auch in der Zielumgebung auf einem Linux-basierten System betrieben werden **?** Zusätzlich unterstützt .NET die containerbasierte Entwicklung und Bereitstellung, was zur bestehenden Infrastruktur des Unternehmens passt.

Da .NET 10 eine aktuelle LTS-Version ist, eignet sich das Framework außerdem für eine langfristig wartbare Anwendung. Durch den großen Funktionsumfang und verfügbare NuGet-Pakete, beispielsweise für Datenbankzugriffe oder Resilienzmechanismen, können die Anforderungen des Projektes ohne den Einsatz weiterer Programmiersprachen umgesetzt werden.

4.3 Datenbankstruktur

Die Datenstruktur liegt zur Veranschaulichung im Folgenden als ERD vor. Sie besteht aus zwei Tabellen *SyncHistory* und *UserContactState*, welche in einer 1:n Beziehung zu einander stehen. Der *UserContactState* beschreibt einen Nutzer, welcher mit *HubSpot* verknüpft wird und die *SyncHistory* beschreibt alle Aktivitäten, die dieser Nutzer im Bezug auf dessen Daten gemacht hat.

Siehe Anhang A.5: Datenbankmodell auf Seite v für das Entity-Relationship-Diagramm (ERD) zur *nightcrawler* Datenbank, welches Attribute, Relationen und die dazugehörigen Multiplizitäten enthält.

5 Implementierung

Dieser Abschnitt beschreibt, wie bei der Erstellung der Anwendung vorgegangen wurde. Zuerst wurden die Datenbanksturkturen und Anbindung implementiert. Danach wurde der Microservice erstellt. Anschließend wurden die Message Broker Abhängigkeiten und der HTTP-Client für die Verbindung zum CRM eingebaut. Schließlich wurden die Datenbankanfragen bzw. Synchronisationshistorie und die Geschäftslogik entwickelt.

5.1 Implementierung der Datenstrukturen

Die PostgreSQL Datenbank *nightcrawler* wurde händisch über *pgAdmin 4* angelegt und nutzt die *libpq-15.dll* Bibliothek. Im Anschluss wurde mit Hilfe des objekt-relationalen Mappers (ORM) EFCore eine Migration angelegt, die die Tabellen, Eigenschaften und Beziehungen zwischen Entitäten definiert

Microsoft [2026]. Diese Migration wurde auf die Datenbank angewandt. Mit diesem Code-First-Ansatz liegt die Kontrolle über die Datenbankstruktur im versionierten Code.

5.2 Implementierung RabbitMQ

Für die Verbindung zwischen dem Microservice *Nightcrawler* und dem Message Broker *RabbitMQ* wurde mithilfe von Serviceregistrierung in der DI-Klasse eine IConnectionFactory als Singleton mit den Applikationseinstellungen registriert. Durch die Registrierung als Singleton können alle Klassen, die eine Verbindung zu *RabbitMQ* benötigen, mit Dependency Injection dieselbe Instanz erzeugen. Dadurch müssen nicht alle Klassen die Einstellungen kennen, um die richtige *RabbitMQ* Instanz anzusprechen.

Um die verschiedenen Queues zu initialisieren wurde ein QueueInitializer erstellt, welcher als IHostedService implementiert wird, um beim Start der Applikation die Queues zu erstellen bzw. zu versichern, dass sie vorhanden sind. Innerhalb dieses Services wird die Topologie für *RabbitMQ* verwaltet. Hier werden die MainQueues, RetryQueues und Dead-Letter-Queue (DLQ)'s erstellt, die später für den Eventverkehr und das Errorhandling benötigt werden.

Als letztes werden EventPublisher und EventConsumer benötigt, um Events senden und empfangen zu können. Hierzu wurde im *user-service* auf Infrastruktur-Ebene ein EventPublisher implementiert, welcher indirekt nach einem API-Aufruf, die zu synchronisierenden Daten als Event sendet. Der *Nightcrawler* beinhaltet auf Presentations-Ebene den EventConsumer und empfängt die gesendeten Daten, um diese dann zu verarbeiten und mit *HubSpot* zu synchronisieren.

Screenshots der Anwendung in Bezug auf *RabbitMQ* befinden sich im Anhang A.6: RabbitMQ auf Seite vi.

5.3 Implementierung des HTTP-Clients

Um nach dem Empfangen eines Events die Synchronisierung in Richtung *HubSpot* anzustoßen wird ein HTTP-Client benötigt. Dafür wurde der HubSpotApiClient implementiert, welcher Daten aus dem Domänen-Modell mappt und damit die Http-Requests ausführt. Dieser Client ermöglicht POST und PATCH Anfragen bzw. das Erstellen und Bearbeiten von Kontaktdatensätzen in *HubSpot* HubSpot, Inc. [2026].

Damit die Anfragen die korrekte *HubSpot* Instanz erreicht benötigt es eine *BaseUrl*, also den ersten Teil einer URL, wodurch die Instanz angesprochen wird. Desweiteren wird der Kontaktpfad benötigt, also der Ort, an dem die Kontaktdaten verändert werden können. *HubSpot* benötigt für ihre Schnittstellen eine Authorisierung, weshalb der Authorization Header mit Bearer und einem Zugriffstoken für diese HTTP-Anfragen zusätzlich benötigt wird.

Um schließlich alle Anforderungen zu erfüllen wurde ein HubSpotRateLimiter implementiert, der dafür sorgt, dass die von *HubSpot* festgelegten Anfragebegrenzungen nicht überschritten werden und somit keine Anfragen unnötig fehlschlagen oder verloren gehen.

Ausschnitte zum HubSpotApiClient und HubSpotRateLimiter sind im Anhang A.7: HTTP-Client auf Seite vii zu finden.

5.4 Implementierung der Synchronisationshistorie

Um die Synchronisationshistorie dauerhaft speichern und gleichzeitig eine gute Testbarkeit gewährleisten zu können, wurde die Implementierung in drei Abstraktionsebenen aufgeteilt. Diese Trennung sorgt dafür, dass fachliche Logik, technische Verarbeitung und direkter Datenbankzugriff voneinander getrennt bleiben. Abbildung 12 zeigt den vereinfachten Aufbau dieser Ebenen.

Die *DbOperations* befinden sich auf der niedrigsten Schicht und sie können einfache Abfragen auf Datenbankebene machen. Außerdem sind neben Testklassen, die einzigen Klassen, die den *DbContext* kennen.

Bei den *Repositories* handelt es sich, um Klassen, die *DbOperations* implementieren und neben diesen Aufrufen Domänen-Models zu Infrastruktur-Models konvertieren. Zudem befindet sich das Logging, Retry- und Errorhandling auf dieser Ebene. Für die Retrylogik wurde das NuGet Paket *Polly* verwendet, welches ermöglicht vorkonfigurierte *ResiliencePipelines* zu implementieren, mit denen man fehlerhafte Aufrufe für eine bestimmte Anzahl an Wiederholungen erneut ausführen kann Polly [2026].

Die Services beinhalten *Repository*-Aufrufe, sowie minimale Logik, mit der *Upserts* ermöglicht werden. So überprüft man für die *SyncHistory* zum Beispiel anhand der *EventId*, ob es bereits eine Historie gibt. Wenn es keine gibt wird eine erstellt, andernfalls wird sie nur aktualisiert.

Siehe Anhang A.8: Synchronisationshistorie auf Seite viii für Code-Ausschnitte.

5.5 Implementierung der Geschäftslogik

Für die Umsetzung der eigentlichen Geschäftslogik wurden drei weitere Services auf Applikationsebene implementiert, damit auch hier die Testbarkeit und das *single responsibility principle* eingehalten wird Clean Code Developer [2026]. Zwei Services für die Ressourcen *PersonalInformation* und *CommunicationPreference* werden, je nach erhaltener Nachricht im EventConsumer instanziiert und starten den Synchronisationsprozess, der jeweilig deserialisierten Nachricht. Desweiteren beinhalten diese Services die Logik der Erstellung von Synchroinsationshistorien.

Als letztes wurde der HubSpotContactService entwickelt. Dieser beinhaltet den HubSpotApiClient und führt die beiden genannten Ressourcen ein mapping durch, womit diese in eine für *HubSpot* lesbare Form gebracht werden. Fakt ist, dass ein Kontakt nur erstellt werden soll, wenn die Ressource *PersonalInformation* das erste Mal synchronisiert wird. Aus diesem Grund wurden in dieser Klasse Methoden für das Erstellen und Aktualisieren von Kontakten implementiert. Wenn die *contactId*, also die Identifikationsnummer für einen *HubSpot* Kontakt null beträgt, wird ein Kontakt erstellt. Andernfalls wird der Kontakt mit der vorhandenen *contactId* aktualisiert. Für den Fall, dass ein Kontakt mit der gegebenen E-Mail bereits in *HubSpot* existiert, wurde ein *RegEx* eingebaut, welcher anhand des Inhalts, im *Json* Format einer API Antwort mit Statuscode 409, die existierende Indentifikationsnummer extrahiert und zurückgibt. Die *contactId* wird schließlich in dem *UserContactState* gespeichert, wodurch bei erneuter Datenänderung des Nutzers, nurnoch die Daten aktualisiert werden.

Siehe Anhang A.9: Geschäftslogik auf Seite ix

6 Abnahmephase

Um zu versichern, dass Logik und Systemkomponenten wie gefordert funktionieren wurden der Applikation *Nightcrawler* Unit- sowie Integrationstests hinzugefügt. Welche ebenfalls bei jedem push in einen pull-request durch pipelines ausgeführt werden, um nach jeder Änderung zu gewährleisten, dass die Logik sich nicht geändert hat. Beim Framework zum Testen handelt es sich um *NSubstitute* mit *xUnit* und der Erweiterung *FluentAssertions*, was ermöglicht Tests in lesbarer Form im Arrange, Act und Assert Schema zu schreiben. Außerdem wurden die Tests immer nach dem Muster 'zu testende public Methode_Zustand_Resultat' benannt, um damit einheitliche und verständliche Tests zu gewährleisten.

6.1 Unittests

Pro .NET Projekt wurde ein neues Testprojekt für Unittests erstellt. Diese Art von Tests beschränken sich auf das Überprüfen von einzelnen Codeeinheiten. In diesem Fall wurde eine Unittestklasse pro zu testender Klasse erstellt. So erhält jede Testklasse eine echte Instanz der zu testenden Klasse, die aber mit substituierten/gemockten Abhängigkeiten instanziiert werden, sodass sich beim Testen rein auf die zu testende Logik beschränkt werden konnte.

6.2 Integrationstests

Integrationstest bilden den realen Ablauf von Programmen ab. Um echte Kommunikationswege testen zu können wurden die Pakete *Testcontainers.PostgreSql* und *Testcontainers.RabbitMQ* verwendet, die es ermöglichen Instanzen der jeweiligen Technologie mit Docker zu starten, welche in den Integrationstest, wie eine echte Instanz angesprochen werden können.

Im Anhang finden sich die Projektstruktur und Testbeispiele Anhang A.10: Tests auf Seite x.

6.3 Test Ergebnisse

Das Ergebnis dieser Tests ist das Erzielen einer Testabdeckung von 95 Prozent und einer fehlerfreien Anwendung.

Symbol	Coverage (%)	Uncovered/Total Stmts.
Total	95%	112/2471
Infrastructure.PostgreSql	100%	0/156
Domain	100%	0/47
Tests	98%	20/1257
Application	95%	12/254
IntegrationTests	94%	11/190
Infrastructure.PostgreSql.Migrations	92%	13/153
Infrastructure.RabbitMQ	91%	13/140
Presentation.RabbitMQ	87%	24/185
Infrastructure.HubSpot	82%	15/85
Orchestrator	0%	4/4

Anhang A.10: Tests auf Seite x

7 Deployment

Rahmenbedingung des Projektes ist nicht die Aufnahme der Software in den Livebetrieb, jedoch in das Testsystem. Aus diesem Grund wurden GitHub Pipelines implementiert, die alle Testpfade und eine statische Codeanalyse mit *SonarQube* ausführen und bei erfolgreicher Reviewphase eines pull-requests, die Änderungen automatisiert in den main branch des GitHub Repositories mergen. Gleichzeitig bedeutet dies, dass die Software auf einem Linux Debian basierten Server deployed und gehosted wird. Denn die Pipelines führen, nach bestehen aller Tests und des SonarQube Quality Gates, ein Deployment zu dem Testserver über SSH aus.

Im Anhang sind Ausschnitte der yml Dateien zu finden Anhang A.11: Pipelines und Automatisierung auf Seite xiii

8 Dokumentation

In diesem Abschnitt wird auf die im Rahmen des Projektes erstellte und ergänzte Dokumentation eingegangen.

8.1 Entwicklerdokumentation

Da die Anwendung nicht direkt von Endbenutzern bedient wird, ist keine klassische Anwenderdokumentation erforderlich. Stattdessen wurde eine Entwicklerdokumentation in Form einer *README.md* erstellt. Diese richtet sich an Entwickler, die die Anwendung lokal starten, konfigurieren oder erweitern möchten.

Die *README.md* enthält die Voraussetzungen für den erfolgreichen Start der Anwendung, eine Quick-Start-Anleitung, die Projektstruktur, eine Übersicht der verwendeten Konfigurationseinstellungen und Umgebungsvariablen sowie eine Anleitung zum Starten der Anwendung mit Docker Compose Docker Inc. [2026].

Zusätzlich zur externen Entwicklerdokumentation wurde die Dokumentation im Quellcode ergänzt. Dazu gehören insbesondere XML-Summaries an zentralen Interfaces und Klassen, da diese Bestandteil der unternehmensinternen Quellcode-Standards sind. Dadurch können Entwickler die Verantwortlichkeiten wichtiger Komponenten direkt im Code nachvollziehen.

Weitere projektbezogene Informationen, wie fachliche Vorüberlegungen und Architekturentscheidungen, wurden im unternehmensinternen Confluence dokumentiert. Diese Dokumente dienen als ergänzende Entscheidungsgrundlage, wurden jedoch nicht vollständig im Rahmen dieser Projektdokumentation erstellt. Die im Projekt entstandene Anwendung wurde außerdem in die unternehmensinterne Architekturübersicht aufgenommen, damit der Service nach Projektabschluss weiterhin auffindbar und in der Systemlandschaft nachvollziehbar bleibt.

Ein Ausschnitt aus der erstellten Entwicklerdokumentation befindet sich im Anhang A.12: Entwicklerdokumentation auf Seite xiv.

9 Fazit

In diesem Abschnitt geht es darum den Abschluss des Projektes sachlich zu bewerten und transparent darzustellen, inwiefern die Projektplanung eingehalten werden konnte. Außerdem wird gezeigt, was der Autor gelernt hat und wie sich das im Projekt behandelte System in Zukunft weiterentwickeln wird.

9.1 Soll-/Ist-Vergleich

Das Projektziel die dezentralisierte Verfügbarkeit von Daten zweier Kundendaten speichernder Systeme aufzulösen und somit eine Source of Truth zu schaffen wurde erreicht. Die Funktionalität und Einhaltung der Anforderungen wurde somit erfüllt.

Bei der Einhaltung der Projektplanung ist es im Hinblick auf Zeit und Kosten zu Abweichungen gekommen. So wurde das Zeitlimit, um fünf Stunden überschritten und die Kosten sind dazu proportional um 175 Euro gestiegen. Grund ist, dass die Umstellung vom im Projektantrag erwähnten Hangfire-System auf ein EventQueue-basiertes System wie RabbitMQ neue Herausforderungen mit sich brachte. Zum einen war die Technologie im Unternehmen bisher nicht im Einsatz, wodurch ein erhebliches mehr in Einarbeitung geleistet werden musste. Zum anderen wurde klar, dass die Implementierung in diesem Prototypen als Beispielimplementierung für andere unternehmensinterne Projekte dienen kann, weswegen das Projekt mehr Review-Zyklen als üblich unterstand.

Wie in Tabelle 3 zu erkennen ist, konnte die Zeitplanung nicht eingehalten werden.

Phase	Geplant	Tatsächlich	Differenz
Analyse	8 h	8 h	
Entwurf	6 h	6 h	
Implementierung	49 h	54 h	+5 h
Deployment	7 h	7 h	
Dokumentation	10 h	10 h	
Gesamt	80 h	85 h	

Tabelle 3: Soll-/Ist-Vergleich

9.2 Lessons Learned

Der Prüfling hat bei der Durchführung des Projektes gelernt realistischere Einschätzungen zu machen in Bezug auf die Zeitplanung bei bisher unbekannten Technologien. Außerdem ist dem Prüfling nun der Umgang mit RabbitMQ, den einhergehenden Topologien und Einstellungen für Queues vertraut. Zudem kann der Prüfling zukünftlich für Fragen in diesem Bereich zur Seite stehen und anderen Entwicklern bei der Implementierung helfen.

9.3 Ausblick

Im Rahmen des Projektes wurde die Anzahl der zu synchronisierenden Datenfelder bewusst begrenzt. Zukünftig kann die Anwendung erweitert werden, sodass weitere kundenspezifische Daten zwischen den beteiligten Systemen synchronisiert werden. Dadurch könnte der manuelle Pflegeaufwand in weiteren Prozessen zusätzlich reduziert werden.

Eine weitere mögliche Erweiterung besteht darin, die gespeicherte Synchronisationshistorie gezielter auszuwerten. Über eine interne Schnittstelle oder ein Dashboard könnten beispielsweise fehlgeschlagene Synchronisationen, wiederholte Verarbeitungsversuche oder häufig auftretende Konflikte sichtbar gemacht werden. Dadurch könnten technische Fehler schneller erkannt und die Datenqualität langfristig besser überwacht werden.

Literaturverzeichnis

Broadcom Inc. 2026
Broadcom Inc.: *RabbitMQ Documentation*. https://www.rabbitmq.com/docs. Version: 2026, Abruf: 07.05.2026

Clean Code Developer 2026
Clean Code Developer: *Die Grade*. https://clean-code-developer.de/die-grade/. Version: 2026, Abruf: 07.05.2026

Docker Inc. 2026
Docker Inc.: *Docker Compose*. https://docs.docker.com/compose/. Version: 2026, Abruf: 07.05.2026

HubSpot, Inc. 2026
HubSpot, Inc.: *API Reference Overview*. https://developers.hubspot.com/docs/api-reference/latest/overview. Version: 2026, Abruf: 07.05.2026

Microsoft 2026
Microsoft: *Overview of Entity Framework Core*. https://learn.microsoft.com/en-us/ef/core/. Version: 2026, Abruf: 07.05.2026

Polly 2026
Polly: *Resilience pipelines*. https://www.pollydocs.org/pipelines/index.html. Version: 2026, Abruf: 07.05.2026

The PostgreSQL Global Development Group 2026
The PostgreSQL Global Development Group: *PostgreSQL 16.13 Documentation*. https://www.postgresql.org/docs/16/index.html. Version: 2026, Abruf: 07.05.2026

A Anhang

A.1 Detaillierte Zeitplanung

Analyse		**8 h**
1. Ist-Analyse durchführen	2 h	
2. Wirtschaftlichkeitsprüfung	2 h	
3. Anforderungsanalyse	1 h	
4. Nutzwertanalyse	3 h	
Entwurf		**6 h**
1. UML-Aktivitätsdiagramm	2 h	
2. Architekturübersicht	2 h	
3. Entwurf von Datenbankstruktur und Tabellen	2 h	
Implementierung		**49 h**
1. Repo/Projektsetup	1 h	
2. HTTP-Clients	7 h	
3. Authentifizierung	1 h	
4. Anbindung Hangfire	5 h	
5. Implementierung Sync-Service	2 h	
5.1. Implementierung Sync-Repository	10 h	
5.2. Implementierung Sync-DbOperations	4 h	
5.3. Implementierung Hangfire Backgroundjobs	3 h	
6. Mapping/Validierung	1 h	
7. Anbindung/Erstellung Datenbank	3 h	
8. Errorhandling/Logging/Retry	5 h	
9. Unit-/Integrationstests	7 h	
Deployment		**7 h**
1. Einbindung Dockerfiles	1 h	
2. Konfiguration GitHub Secrets/Variables	1 h	
3. CI/CD Pipelines	4 h	
4. Deployment C#-Microservice	1 h	
Dokumentation		**10 h**
1. Erstellen der Projektdokumentation	8 h	
2. Erstellen der Entwicklerdokumentation	2 h	
Gesamt		**80 h**

A.2 Jira-Tickets

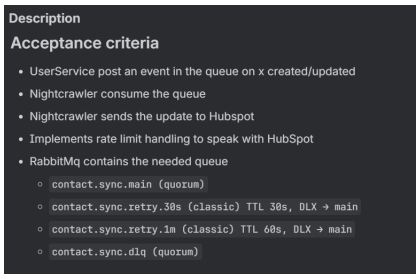


Abbildung 1: Ticket Synchronisation

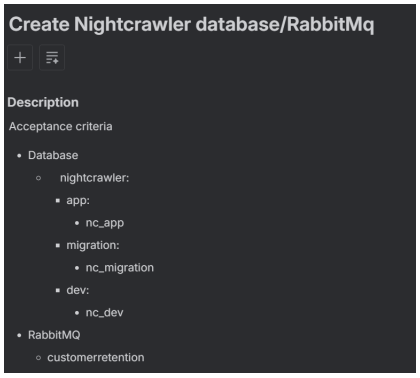


Abbildung 2: Ticket Datenbank und RabbitMQ

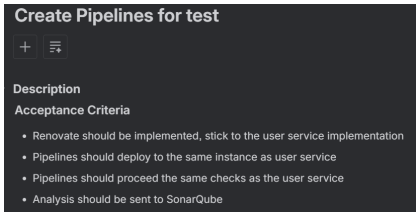
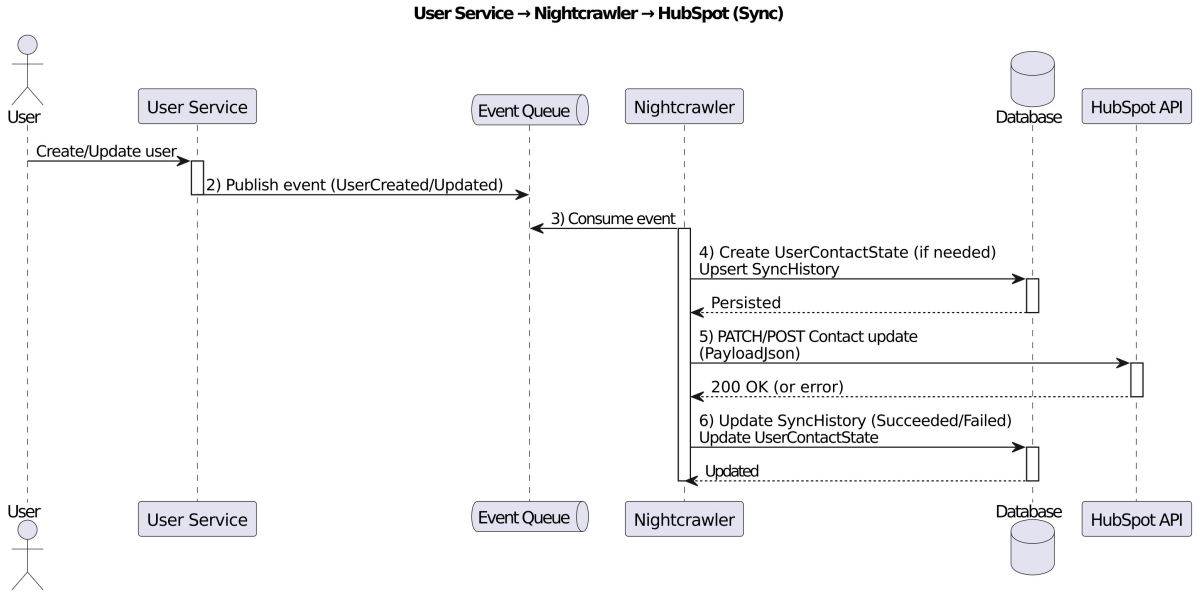


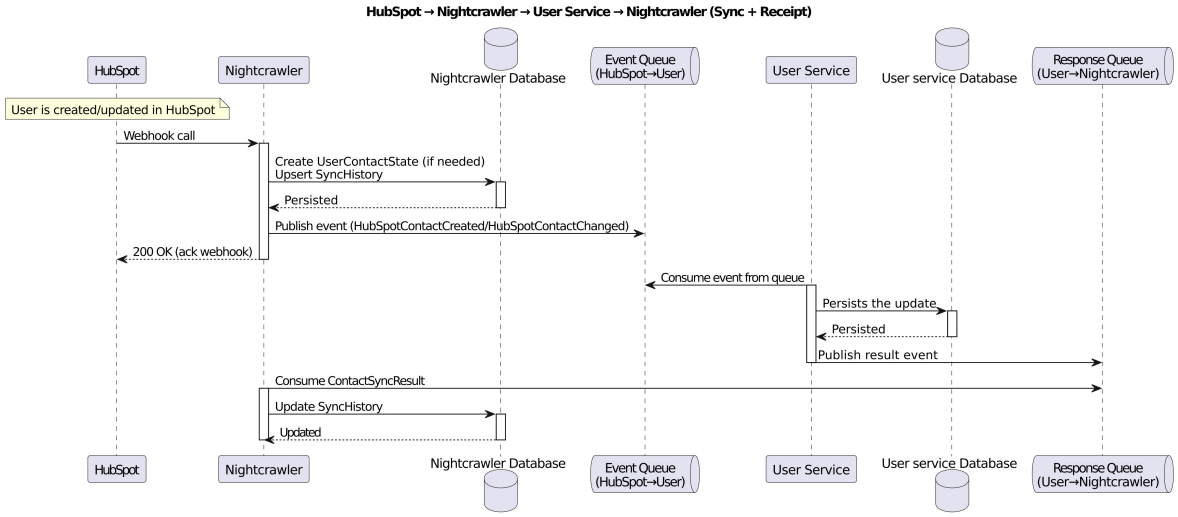
Abbildung 3: Ticket Pipelines

A.3 Synchronisations Abläufe

User-Service zu HubSpot



HubSpot zu User-Service



A.4 Architektur

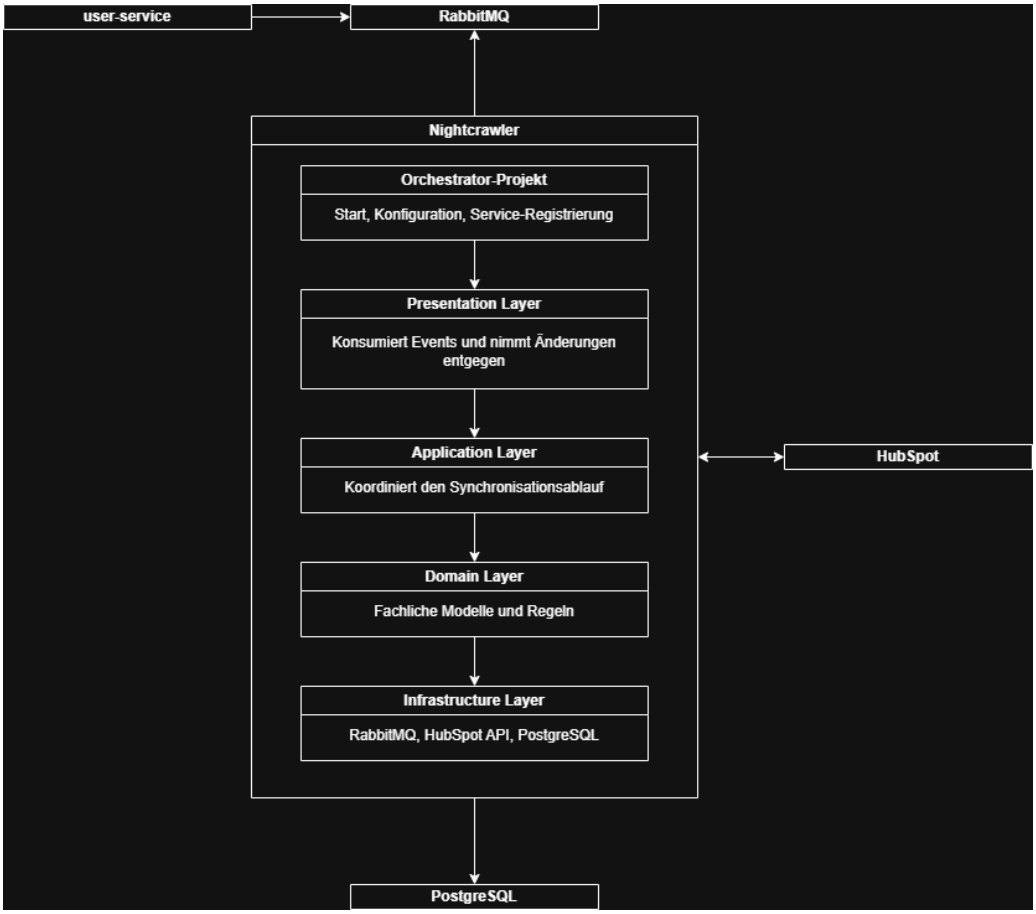


Abbildung 4: Vereinfachte Architekturübersicht der Nightcrawler-Solution mit Orchestrator-Projekt und Anwendungsschichten

A.5 Datenbankmodell

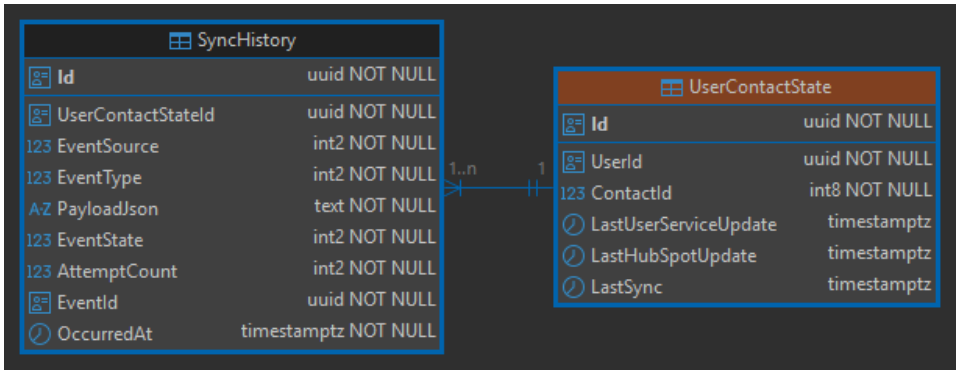


Abbildung 5: Nightcrawler ER-Diagramm

A.6 RabbitMQ

```
private static IServiceCollection AddRabbitMqConnectionDependencies(this IServiceCollection services)
{
    services.AddSingleton<IConnectionFactory>(serviceProvider =>
    {
        ConnectionOptions connectionOptions = serviceProvider.GetRequiredService<IOptions<ConnectionOptions>>().Value;

        ConnectionFactory connectionFactory = new()
        {
            HostName = connectionOptions.HostName,
            Port = connectionOptions.Port,
            UserName = connectionOptions.UserName,
            Password = connectionOptions.Password
        };

        return connectionFactory;
    });

    return services;
}
```

Abbildung 6: RabbitMQ DI-Registrierung

```
public async Task StartAsync(CancellationToken cancellationToken)
{
    try
    {
        await using IConnection connection =
            await _connectionFactory.CreateConnectionAsync(cancellationToken);

        await using IChannel channel =
            await connection.CreateChannelAsync(cancellationToken: cancellationToken);

        await channel.ExchangeDeclareAsync(
            exchange: _queueOptions.ExchangeName,
            type: ExchangeType.Direct,
            durable: true,
            autoDelete: false,
            cancellationToken: cancellationToken);//Task

        await DeclareMainQueueAsync(channel, cancellationToken);
        await BindQueueAsync(
            channel,
            _queueOptions.QueueNameMain,
            _queueOptions.MainRoutingKey,
            cancellationToken);

        await DeclareRetry30SQueueAsync(channel, cancellationToken);
        await DeclareRetry1MQueueAsync(channel, cancellationToken);

        await DeclareDeadLetterQueueAsync(channel, cancellationToken);
        await BindQueueAsync(
            channel,
            _queueOptions.QueueNameDlq,
            _queueOptions.DeadRoutingKey,
            cancellationToken);

        _logger.LogInformation("Queues initialized successfully!");
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, message: "An error occurred while initializing the queues.");
    }
}
```

Abbildung 7: Queue-Initialisierung

```
public async Task PublishAsync<TMessage>,
    EventMessage<TMessage> message,
    CancellationToken cancellationToken)
{
    string serializedMessage = JsonSerializer.Serialize(message);
    byte[] body = Encoding.UTF8.GetBytes(serializedMessage);

    BasicProperties properties = new()
    {
        Persistent = true,
        ContentType = "application/json",
        Type = EventNetMapper.GetEventTypeId(),
        MessageId = message.EventId.ToString()
    };

    await _lock.WaitAsync(cancellationToken);
    try
    {
        await _resiliencePipeline.ExecuteAsync(
            async (ct => await PublishInternalAsync(properties, body, ct),
            cancellationToken).ConfigureAwait();

        if (_logger.IsEnabled(LogLevel.Information))
        {
            _logger.LogInformation(
                "Successfully published message {MessageId} of type {Type} to exchange {Exchange} with routing key {RoutingKey}",
                properties.MessageId,
                properties.Type,
                _queueOptions.ExchangeName,
                _queueOptions.RoutingKey);
        }
    }
    catch (PublishException ex) when (ex.IsRetryable)
    {
        _logger.LogError(
            ex,
            exceptionMessage: "Message {MessageId} of type {Type} was returned to broker and will not be retried. Exchange: {Exchange}, RoutingKey: {RoutingKey}",
            properties.MessageId,
            properties.Type,
            _queueOptions.ExchangeName,
            _queueOptions.RoutingKey);

        throw new EventPublishException(
            $"Publish failed for message {properties.MessageId} of type {properties.Type}", ex);
    }
    catch (Exception ex)
    {
        _logger.LogError(
            ex,
            exceptionMessage: "Failed to publish message {MessageId} of type {Type}",
            properties.MessageId,
            properties.Type);

        throw new EventPublishException(
            $"Publish failed for message {properties.MessageId} of type {properties.Type}", ex);
    }
    finally
    {
        _lock.Release();
    }
}
```

Abbildung 8: Event Publisher

```
protected override async Task ExecuteAsync(CancellationToken stoppingToken)
{
    await EnsureInitializedAsync(stoppingToken);

    await _channel!.BasicQosAsync(
        prefetchSize: 0,
        prefetchCount: 1,
        global: false,
        cancellationToken: stoppingToken);//Task

    AsyncEventingBasicConsumer consumer = new(_channel);
    consumer.ReceivedAsync += async (_, eventArgs)
        => await OnReceivedAsync(eventArgs, stoppingToken);

    _consumerTag = await _channel.BasicConsumeAsync(
        queue: _queueOptions.QueueNameMain,
        autoAck: false,
        consumer: consumer,
        cancellationToken: stoppingToken);//Task<string>

    if (_logger.IsEnabled(LogLevel.Information))
    {
        _logger.LogInformation(
            "Started consuming queue {QueueName} with consumer tag {ConsumerTag}",
            _queueOptions.QueueNameMain,
            _consumerTag);
    }

    try
    {
        await Task.Delay(Timeout.InFinite, stoppingToken);
    }
    catch (OperationCanceledException ex)
    {
        if (_logger.IsEnabled(LogLevel.Information))
        {
            _logger.LogInformation(
                ex,
                message: "Stopping RabbitMQ consumer for queue {QueueName}.",
                _queueOptions.QueueNameMain);
        }
    }
}
```

Abbildung 9: Event Consumer

A.7 HTTP-Client

```
public class HubSpotApiClient : IHubSpotApiClient
{
    private const string s_contactsPath = "/crm/v3/objects/contacts";

    private readonly HttpClient _httpClient;
    private readonly IHubSpotRateLimiter _rateLimiter;
    private readonly IJsonContentMapper _mapper;

    1 usage   Nhamburg.Journaway
    public HubSpotApiClient(
        IHttpClientFactory httpClientFactory,
        IHubSpotRateLimiter rateLimiter,
        IOptions<HubSpotOptions> options,
        IJsonContentMapper mapper)
    {
        HubSpotOptions hubSpotOptions = options.Value;
        _httpClient = httpClientFactory.CreateClient(nameof(HubSpotApiClient));
        _rateLimiter = rateLimiter;
        _mapper = mapper;
        _httpClient.BaseAddress = new Uri(hubSpotOptions.BaseUrl);
        _httpClient.DefaultRequestHeaders.Authorization =
            new AuthenticationHeaderValue(scheme: "Bearer", hubSpotOptions.AccessToken);
    }

    2×12 usages   2 tests OK   Nhamburg.Journaway
    public async Task<HttpResponseMessage> PostAsync(HubSpotPostRequest requestBody, CancellationToken cancellationToken)
    {
        await _rateLimiter.WaitForAvailabilityAsync(cancellationToken);

        using HttpRequestMessage requestMessage = new(
            HttpMethod.Post,
            requestUri: s_contactsPath);
        requestMessage.Content = _mapper.ToJsonContent(requestBody);

        HttpResponseMessage response = await _httpClient.SendAsync(requestMessage, cancellationToken);

        _rateLimiter.UpdateFromHeaders(response.Headers);
        return response;
    }

    1×15 usages   Nhamburg.Journaway
    public async Task<HttpResponseMessage> PatchAsync(HubSpotPatchRequest requestBody, long contactId, CancellationToken cancellationToken)
    {
        await _rateLimiter.WaitForAvailabilityAsync(cancellationToken);

        string path = $"{s_contactsPath}/{contactId}";

        using HttpRequestMessage requestMessage = new(HttpMethod.Patch, path);
        requestMessage.Content = _mapper.ToJsonContent(requestBody);

        HttpResponseMessage response = await _httpClient.SendAsync(requestMessage, cancellationToken);

        _rateLimiter.UpdateFromHeaders(response.Headers);
        return response;
    }
}
```

Abbildung 10: HubSpotApiClient

```
public async Task WaitForAvailabilityAsync(CancellationToken cancellationToken)
{
    while (true)
    {
        TimeSpan delay;

        await _gate.WaitAsync(cancellationToken);
        try
        {
            DateTimeOffset now = DateTimeOffset.UtcNow;
            TrimExpired(now);

            if (_requestTimestamps.Count < _maxRequestsPerWindow)
            {
                _requestTimestamps.Enqueue(now);
                return;
            }

            DateTimeOffset oldest = _requestTimestamps.Peek();
            DateTimeOffset nextAllowed = oldest + _window;
            delay = nextAllowed > now;
        }
        finally
        {
            _gate.Release();
        }

        if (_logger.IsEnabled(LogLevel.Debug))
        {
            _logger.LogDebug(
                "HubSpot throttling for {DelayMs} ms (maxRequests={MaxRequests}, windowMs={WindowMs})",
                (int)delay.TotalMilliseconds,
                _maxRequestsPerWindow,
                (int)_window.TotalMilliseconds);
        }

        await Task.Delay(delay, cancellationToken);
    }
}

2×5 usages   Nhamburg.Journaway
public void UpdateFromHeaders(HttpResponseHeaders headers)
{
    if (!TryGetInt(headers, name: s_hubspotRateLimitMax, out int max) ||
        !TryGetInt(headers, name: s_hubspotRateLimitIntervalMilliseconds, out int intervalMs))
    {
        return;
    }

    if (max <= 0 || intervalMs <= 0)
    {
        return;
    }

    _maxRequestsPerWindow = max;
    _window = TimeSpan.FromMilliseconds(intervalMs);
}
```

Abbildung 11: HubSpotApiRateLimiter

A.8 Synchronisationshistorie

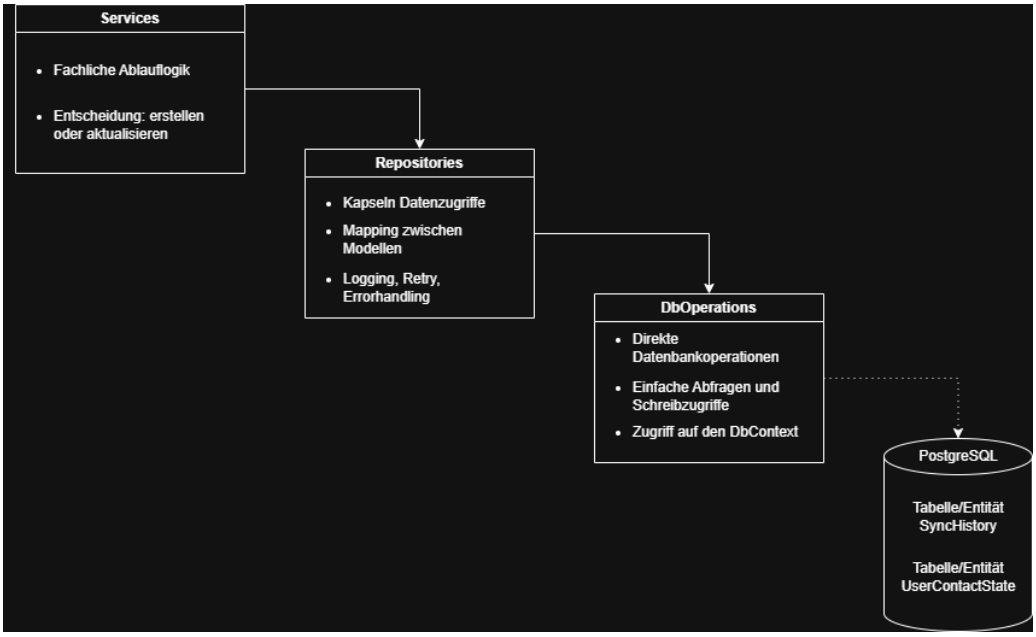


Abbildung 12: Drei Abstraktionsebenen zur Persistierung der Synchronisationshistorie

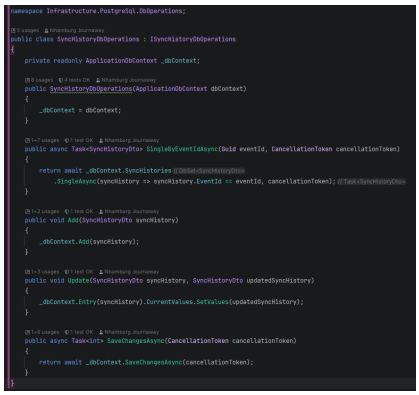


Abbildung 13: DbOperation

Abbildung 14: Repository

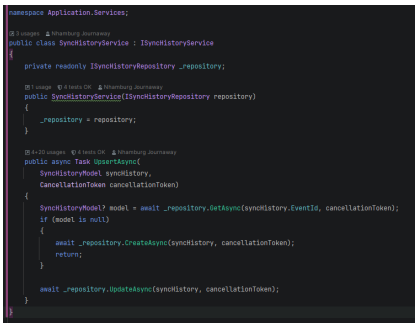


Abbildung 15: Service

A.9 Geschäftslogik

```
9+7 usages   Nhamburg Journaway
public async Task<long> SyncPersonalInformationAsync(
    PersonalInformationChangedPayload payload,
    long contactId,
    CancellationToken cancellationToken)
{
    ArgumentNullException.ThrowIfNull(payload);

    Dictionary<string, object?> properties = BuildPersonalInformationProperties(payload);

    if (contactId is 0)
    {
        return await CreateContactAsync(properties, cancellationToken);
    }

    await UpdateContactAsync(contactId, properties, cancellationToken);

    return contactId;
}
```

Abbildung 16: PersonalInformation Upsert

```
private static Task<long> CreateCommonRecord
    Dictionary<string, object?> properties,
    CancellationToken cancellationToken)
{
    string traceId = Guid.NewGuid().ToString();

    HubSpotPostRequest requestBody = new()
    {
        Properties = properties,
        ObjectWriteTraceId = traceId
    };

    using HttpResponseMessage response = await _apiClient.Post(uri:requestBody, cancellationToken:);
    string responseJson = await response.Content.ReadAsStringAsync(cancellationToken);

    if (response.StatusCode == HttpStatusCode.Conflict)
    {
        long existingContactId = ExtractExistingContactIdFromConflictResponse(responseJson);

        if (_logger.IsEnabled(LogLevel.Information))
        {
            _logger.LogInformation(
                "HubSpot contact already exists. Updating existing contact {contactId:D} {traceId:D}",
                existingContactId,
                traceId);
        }

        await UpdateContactAsync(existingContactId, properties, cancellationToken);

        return existingContactId;
    }

    if (response.StatusCode != HttpStatusCode.Created)
    {
        _logger.LogError(
            "HubSpot create contact failed {statusCode:D} {traceId:D} {response:@response}",
            (int)response.StatusCode,
            traceId,
            responseJson);

        throw new HubSpotTrxException("HubSpot create contact failed");
    }

    HubSpotPostResponse? parsed =
        JsonSerializer.Deserialize<HubSpotPostResponse>(responseJson, s_jsonOptions);

    if (parsed is not null && long.TryParse(parsed.Id, out long contactId))
    {
        if (_logger.IsEnabled(LogLevel.Information))
        {
            _logger.LogInformation(
                "HubSpot created contact successfully {contactId:D} {traceId:D}",
                contactId,
                traceId);
        }

        return contactId;
    }

    _logger.LogError(
        "{contactId} could not be retrieved from response {traceId:D}",
        traceId);

    throw new HubSpotNullException("HubSpot ContactId could not be retrieved from response");
```

Abbildung 17: API-Logik

```
private static long ExtractExistingContactIdFromConflictResponse(string responseJson)
{
    Match match = s_existingContactIdRegex.Match(responseJson);

    if (match.Success && long.TryParse(match.Groups["id"].Value, out long contactId))
    {
        return contactId;
    }

    throw new HubSpotSyncException(
        "HubSpot contact already exists, but ContactId could not be extracted from conflict response");
}

[GeneratedRegex(pattern: @"""Existing ID:\s*?(?<id>\d+)?""", options: RegexOptions.IgnoreCase | RegexOptions.Compiled | RegexOptions.CultureInvariant, matchTimeoutMilliseconds: 100)]
1 usage   Nhamburg Journaway
private static partial Regex ExistingContactIdRegex();
```

Abbildung 18: RegEx-Aufruf

```
public async Task SyncAsync(
    EventMessage<CommunicationPreferenceChangedPayload> message,
    short attempt,
    CancellationToken cancellationToken)
{
    UserId userId = new(message.Payload.UserId);

    UserContactStateModel userContactState =
        await GetUserContactStateAsync(userId, cancellationToken);

    string newJsonJson = JsonSerializer.Serialize(message.Payload, s_jsonOptions);
    SyncHistoryId id = new(Ulid.NewUlid());
    EventId eventId = new(message.EventId);

    SyncHistoryModel syncHistory = new(
        Id: id,
        UserContactStateId: userContactState.Id,
        EventId: eventId,
        EventSource: message.EventSource,
        EventType: message.EventType,
        PayloadJson: newJsonJson,
        OccurredAt: message.OccurredAt.UtcDateTime,
        EventState: EventState.Pending,
        AttemptCount: attempt);

    try
    {
        bool isCreated = await InsertIgnoreOnSyncHistoryAsync(syncHistory, userContactState, cancellationToken);
        if (!isCreated)
        {
            return;
        }

        await OpenPendingSyncHistoryAsync(syncHistory, cancellationToken);

        await _hubSpotContactService.SyncCommunicationPreferenceAsync(message.Payload, userContactState.ContactId, cancellationToken);

        await UpdateUserContactStateAsync(message.OccurredAt, userContactState, cancellationToken);

        await UpdateSuccessfulSyncHistoryAsync(syncHistory, cancellationToken);
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, message: "Unexpected error occurred while processing '{EventId}'", message.EventId);

        await OpenFailedSyncHistoryAsync(syncHistory, cancellationToken);

        throw new CommunicationPreferenceSyncException(
            $"An error occurred while syncing CommunicationPreference {ex.Message}", ex);
    }
}
```

Abbildung 19: CommunicationPreference-Logik

A.10 Tests

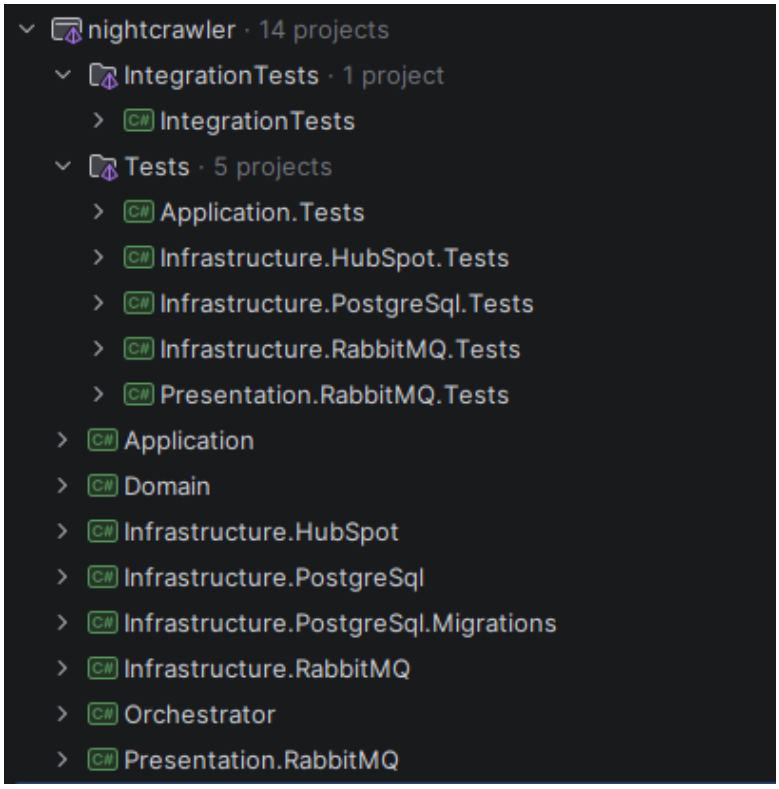


Abbildung 20: Nightcrawler Projekt Struktur

Symbol	Coverage (%)	Uncovered/Total Stmts.
Total	95%	112/2471
Infrastructure.PostgreSql	100%	0/156
Domain	100%	0/47
Tests	98%	20/1257
Application	95%	12/254
IntegrationTests	94%	11/190
Infrastructure.PostgreSql.Migrations	92%	13/153
Infrastructure.RabbitMQ	91%	13/140
Presentation.RabbitMQ	87%	24/185
Infrastructure.HubSpot	82%	15/85
Orchestrator	0%	4/4

Abbildung 21: Testabdeckung

```
[Fact]
1 test OK   Nhamburg Journaway
public async Task SyncCommunicationPreferenceAsync_WithValidPayload_UpdatesContact()
{
    // Arrange
    CancellationToken cancellationToken = CancellationToken.None;

    CommunicationPreferenceChangedPayload payload = CreateCommunicationPreferencePayload(
        isNewsletterEnabled: true);

    _apiClient // IHubSpotApiClient
        .PatchAsync(
            requestBody: Arg.Any<HubSpotPatchRequest>(),
            contactId: Arg.Any<long>(),
            cancellationToken: Arg.Any<CancellationToken>()) // Task<HttpResponseMessage>
        .Returns( returnThis: CreateResponse(HttpStatusCode.OK));

    // Act
    await _sut.SyncCommunicationPreferenceAsync(
        payload,
        contactId: 123,
        cancellationToken); // Task

    // Assert
    await _apiClient.Received(1).PatchAsync(
        requestBody: Arg.Is<HubSpotPatchRequest>(request =>
            request.Properties["ja_userid"]!.Equals(payload.UserId.ToString()) &&
            request.Properties["agreement_newsletter"]!.Equals(true) &&
            !string.IsNullOrWhiteSpace(request.ObjectWriteTraceId)),
        contactId: 123,
        cancellationToken); // Task<HttpResponseMessage>
}
```

Abbildung 22: Unit Test

```
[Fact]
1 test OK   Nhamburg Journaway
public async Task Add_SyncHistoryWasAdded_DbContextContainsAddedSyncHistory()
{
    // Arrange
    Guid syncHistoryId = Guid.NewGuid();
    Guid userContactStateId = Guid.NewGuid();

    SyncHistoryDto syncHistoryDto = new()
    {
        Id = syncHistoryId,
        UserContactStateId = userContactStateId,
        EventId = Guid.NewGuid(),
        EventSource = EventSource.UserService,
        EventType = EventType.Create,
        PayloadJson = """{"email":"test@example.com"}""",
        OccurredAt = DateTime.UtcNow,
        EventState = EventState.Pending,
        AttemptCount = 1
    };

    await SeedUserContactStateAsync(userContactStateId, CancellationToken.None);

    await using ApplicationDbContext dbContext = _factory.CreateDbContext();
    SyncHistoryDbOperations sut = new(dbContext);

    // Act
    sut.Add(syncHistoryDto);

    // Assert
    dbContext.SyncHistories.Local.Should() // GenericCollectionAssertions<SyncHistoryDto>
        .ContainSingle(syncHistory => syncHistory == syncHistoryDto);
}
```

Abbildung 23: Integrationstest

A.11 Pipelines und Automatisierung

```yaml
jobs:
  run-all-tests:
    steps:
      - name: SonarQube Scan & Run tests
        shell: bash
        env:
          SONAR_TOKEN: ${{ secrets.sonar_token }}
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
          REPOSITORY_OWNER: ${{ github.repository_owner }}
          REPOSITORY_NAME: ${{ github.event.repository.name }}
          HEAD_REF: ${{ github.head_ref }}
          BASE_REF: ${{ github.base_ref }}
          REF_NAME: ${{ github.ref_name }}
          PR_NUMBER: ${{ github.event.pull_request.number }}
        run: |
          set -Eevo pipefail
          IFS=$'\n\t'
          set -f

          SONAR_SCANNER="./.sonar/scanner/dotnet-sonarscanner"

          EXTRA_ARGS=()

          if [[ "${GITHUB_EVENT_NAME}" == "pull_request" ]]; then
            EXTRA_ARGS+=("/d:sonar.pullrequest.base=${BASE_REF}")
            EXTRA_ARGS+=("/d:sonar.pullrequest.branch=${HEAD_REF}")
            EXTRA_ARGS+=("/d:sonar.pullrequest.key=${PR_NUMBER}")
          else
            EXTRA_ARGS+=("/d:sonar.branch.name=${REF_NAME}")
          fi

          dotnet sonarscanner begin \
            /k:"${REPOSITORY_OWNER}_${REPOSITORY_NAME}" \
            /o:"${REPOSITORY_OWNER}" \
            /n:"${REPOSITORY_OWNER}" \
            /d:sonar.token="${SONAR_TOKEN}" \
            /d:sonar.host.url="https://sonarcloud.io" \
            /d:sonar.cs.opencover.reportsPaths=".coverage/**/coverage.opencover.xml" \
            /d:sonar.cs.vstest.reportsPaths=".testresults/**/*.trx" \
            /d:sonar.scanner.skipJreProvisioning=true \
            "${EXTRA_ARGS[@]}"

          dotnet restore --locked-mode --nologo
          dotnet build --no-restore --nologo

          mkdir -p .coverage
          dotnet test \
            --no-build \
            --nologo \
            --logger:"trx" \
            --results-directory .testresults \
            /p:CollectCoverage=true \
            /p:CoverletOutputD="./.coverage/" \
            /p:MergeWith="../.coverage/coverage.json" \
            /p:CoverletOutputFormat="\"json,opencover\"

          dotnet sonarscanner end /d:sonar.token="${SONAR_TOKEN}"
```

Abbildung 24: Tests und Codeanalyse

```yaml
build-and-publish-image:
  needs: [ run-all-tests ]
  runs-on: ubuntu-latest
  permissions:
    contents: write
    packages: write
  outputs:
    version: ${{ steps.semver.outputs.version }}
  steps:
    - name: Checkout Code
      uses: actions/checkout@ee2fmx2e45806ade8009e57314ff5f54c7ce83dd #v4.0.2
      with:
        fetch-depth: 0

    - name: Generate semantic version
      id: semver
      uses: paxhatch/semantic-version@e8f8f59fc7f9b2518b492e9432d5f12d7ae2dca3 #v5.4.0
      with:
        tag_prefix: "v"
        major_pattern: "/!:(BREAKING CHANGE:/"
        minor_pattern: "/feat:/"
        version_format: "v${major}.${minor}.${patch}-beta.${increment}"

    - name: Build and push Orchestrator Docker image
      uses: ./.github/actions/build-docker-image
      with:
        image_name: ${{ vars.DOCKER_IMAGE_ORCHESTRATOR }}
        dockerfile: ./Orchestrator/Dockerfile
        context: .
        push: true
        version_tag: ${{ steps.semver.outputs.version }}
        docker_registry: ${{ vars.DOCKER_REGISTRY }}
        registry_user: ${{ github.actor }}
        registry_owner: ${{ github.repository_owner }}
        registry_password: ${{ secrets.GITHUB_TOKEN }}

    - name: Build and push Infrastructure.PostgreSql.Migrations Docker image
      uses: ./.github/actions/build-docker-image
      with:
        image_name: ${{ vars.DOCKER_IMAGE_MIGRATION }}
        dockerfile: ./Infrastructure.PostgreSql.Migrations/Dockerfile
        context: .
        push: true
        version_tag: ${{ steps.semver.outputs.version }}
        docker_registry: ${{ vars.DOCKER_REGISTRY }}
        registry_user: ${{ github.actor }}
        registry_owner: ${{ github.repository_owner }}
        registry_password: ${{ secrets.GITHUB_TOKEN }}
```

Abbildung 25: Docker Images

```yaml
    - name: Create and push Git tag
      shell: bash
      env:
        VERSION: ${{ steps.semver.outputs.version }}
      run: |
        set -Eevo pipefail
        IFS=$'\n\t'
        set -f

        if git rev-parse -q --verify "refs/tags/$VERSION" >/dev/null 2>&1; then
          echo "Tag $VERSION already exists. Skipping."
          exit 0
        fi

        git tag "$VERSION"
        git push origin "$VERSION"

    - name: Upload docker-compose.yml as artifact
      uses: actions/upload-artifact@ea165f8d65b6e75b540449e92b4886f43607fd02 #v4.6.2
      with:
        name: docker-compose
        path: Docker/docker-compose.yml
        retention-days: 14

deploy:
  needs: [ build-and-publish-image ]
  permissions:
    contents: read
    packages: read
  uses: ./.github/workflows/deploy.yml
  with:
    environment: "test"
    version: ${{ needs.build-and-publish-image.outputs.version }}
  secrets:
    migration_database_username: ${{ secrets.MIGRATION_DATABASE_USERNAME }}
    migration_database_password: ${{ secrets.MIGRATION_DATABASE_PASSWORD }}
    orchestrator_database_username: ${{ secrets.ORCHESTRATOR_DATABASE_USERNAME }}
    orchestrator_database_password: ${{ secrets.ORCHESTRATOR_DATABASE_PASSWORD }}
    ssh_key: ${{ secrets.SSH_PRIVATE_KEY }}
```

Abbildung 26: Versionierung

```yaml
- name: Prepare deployment directory
  shell: bash
  env:
    DEPLOYMENT_FOLDER_NAME: ${{ vars.DEPLOYMENT_FOLDER_NAME }}
  run: |
    set -Eevo pipefail
    IFS=$'\n\t'
    set -f

    fail() {
      echo "ERROR: $1" >&2
      exit 1
    }

    [[ -n "$DEPLOYMENT_FOLDER_NAME" ]] || fail "DEPLOYMENT_FOLDER_NAME must not be empty."

    [[ -f "Docker/docker-compose.yml" ]] || fail "Docker/docker-compose.yml not found."
    [[ -f ".env.Common" ]] || fail ".env.Common not found."
    [[ -f ".env.Orchestrator" ]] || fail ".env.Orchestrator not found."
    [[ -f ".env.Migrations" ]] || fail ".env.Migrations not found."

    rm -rf -- "$DEPLOYMENT_FOLDER_NAME"
    mkdir -p -- "$DEPLOYMENT_FOLDER_NAME"

    cp -- "Docker/docker-compose.yml" "$DEPLOYMENT_FOLDER_NAME/docker-compose.yml"
    cp -- ".env.Common" "$DEPLOYMENT_FOLDER_NAME/.env.Common"
    cp -- ".env.Orchestrator" "$DEPLOYMENT_FOLDER_NAME/.env.Orchestrator"
    cp -- ".env.Migrations" "$DEPLOYMENT_FOLDER_NAME/.env.Migrations"

- name: Deploy via SSH
  uses: ./.github/actions/ssh-deploy
  with:
    ssh_key: ${{ secrets.SSH_KEY }}
    ssh_user: ${{ vars.SSH_USER }}
    host: ${{ vars.HOST_DEPLOYMENT }}
    docker_registry: ${{ vars.DOCKER_REGISTRY }}
    registry_user: ${{ github.actor }}
    registry_password: ${{ secrets.GITHUB_TOKEN }}
    deploy_dir: /docker/${{ vars.SSH_USER }}/deploy
    deployment_folder_name: ${{ vars.DEPLOYMENT_FOLDER_NAME }}
    source_dir: ${{ vars.DEPLOYMENT_FOLDER_NAME }}
```

Abbildung 27: Deployment per SSH

A.12 Entwicklerdokumentation

nightcrawler

Customer contact data synchronization between user-service and HubSpot

- Guideline & Conventions
- Architecture

- .NET 10.0 SDK
- Docker
- PostgreSQL database
  - Local installation
  - Docker image: `postgres:16.0-bookworm`

Quick Start IDE

1. Ensure prerequisites are installed
2. Clone the repository
3. Configure the application using environment variables or appsettings
4. Run migrations: `dotnet run --project Infrastructure.PostgreSql.Migrations`
5. Start the service: `dotnet run --project Orchestrator`

Project Structure

```
nightcrawler/
├── Application/                          # Application services & Business logic
├── Domain/                               # Domain models and interfaces
├── Infrastructure.HubSpot/               # HubSpot implementation
├── Infrastructure.PostgreSql/            # PostgreSQL implementation
├── Infrastructure.PostgreSql.Migrations/ # PostgreSQL migrations (console app)
├── Infrastructure.RabbitMQ/              # RabbitMQ implementation
├── Orchestrator/                         # Application composition and startup
├── Presentation.RabbitMQ/                # RabbitMQ consumer
├── *.Tests/                              # Unit Test projects
└── *.IntegrationTests/                   # Integration Test projects
```

Abbildung 28: Ausschnitt Entwicklerdokumentation